

**SingBid Technical Architecture, System Design
& Flow Document
(Version 1.0)**

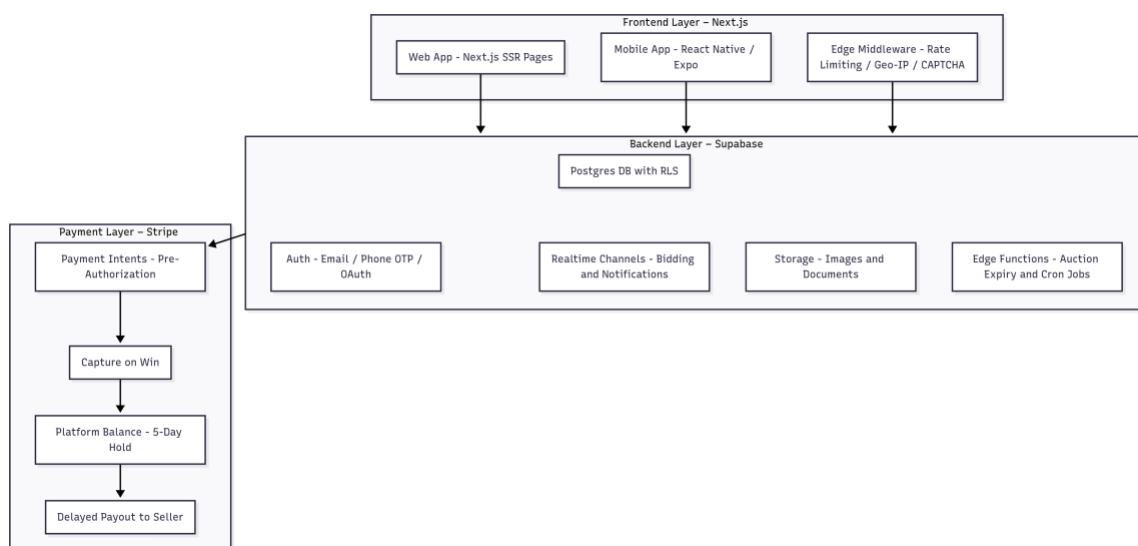
1. System Overview

SingBid is a Singapore-only C2C auction marketplace built on Next.js, Supabase, and Stripe Connect. The platform enables secure and scalable auction functionality with buyer protection, delayed payouts, and compliance with PDPA and PSA requirements. Pages are server-rendered (SSR/ISR) for SEO; payments follow a pre-authorization → capture → platform hold → payout flow.

2. System Architecture Overview

The architecture is composed of three layers:

- Next.js (Frontend + API Routes + Middleware)
 - Supabase (Postgres, Auth, Realtime, Storage, Edge Functions)
 - Stripe Connect (Pre-authorization, Capture, Delayed Payouts)
- Frontend Web (Next.js on Vercel): SSR/prerender for SEO, Edge middleware for rate limiting.
 - Mobile Apps (React Native/Expo): Push notifications for outbid/win/shipping states.
 - API/Server Logic (Supabase Edge Functions): Secure handlers for bids, auctions, disputes, and Stripe webhooks.
 - Database (Supabase Postgres): strict Row Level Security (RLS), audit tables, indexes.
 - Realtime: Supabase Realtime channels for live bids and notifications.
 - Payments: Stripe Connect (Platform), Payment intents with pre-authorization, capture on win, Connect balance hold, delayed payout.
 - Storage: Supabase Storage for media with signed URLs.
 - Jobs/CRON: Edge CRON/Workers for auction expiry, anti-sniping, email, cleanup.
 - Observability: Sentry, GA4, Supabase logs, uptime checks.



3. Detailed Components Breakdown

3.1 Next.js

- Server-side rendering and incremental static regeneration for SEO.
- /api routes for auctions, bidding, payout release and admin operations towards Supabase exposed APIs.
- Webhooks for Stripe payment/dispute handling.
- Middleware for rate limiting, WAF-style checks, and geo/IP gating.

3.2 Supabase

- Postgres with Row-Level Security and audit tables.
- Auth (email, phone OTP, OAuth) with session management.
- Realtime channels for live bidding and notifications.
- Storage for auction images with signed URLs.
- Edge Functions + CRON for auction expiry and anti-sniping.

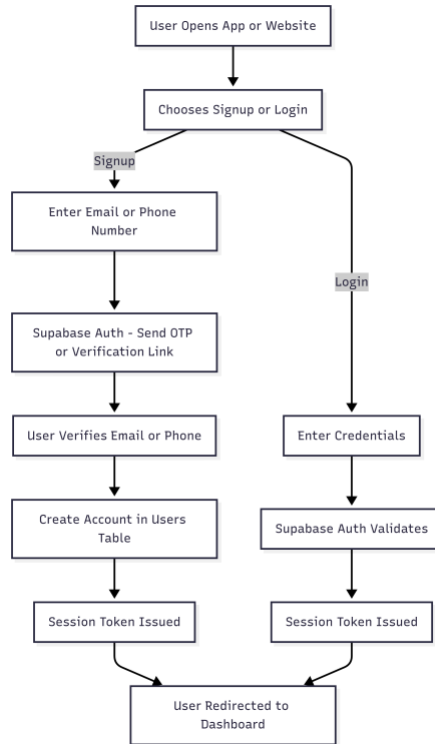
3.3 Stripe Connect

- PaymentIntents with pre-authorization during bidding.
- Capture on win at auction finalization.
- Funds held in platform balance for buyer protection.
- Delayed payout after buyer confirms receipt (or auto after 5 days).
- 5% platform fee retained; remainder paid to seller.

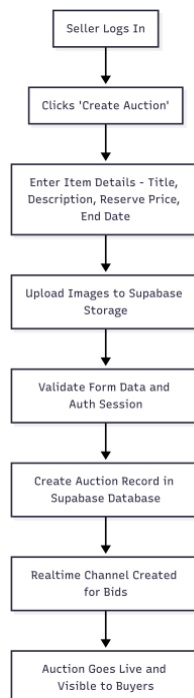
4. Data Flow & Lifecycle

4.1 Key Flows

4.1.1 Registration & Login Auth: phone OTP, optional Google OAuth; later phase: Singpass/MyInfo (GovTech onboarding required).

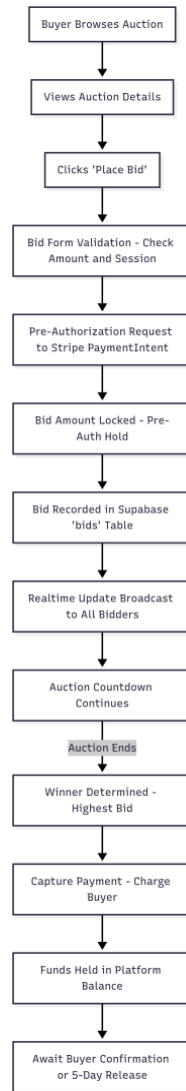


4.1.2 Create Auction: seller lists item, images, reserve, duration, prohibited-item checks.



4.1.3 Place Bid (Realtime):

- Client requests bid; server-side function validates auction status and current max bid atomically.
- If valid, write bid row within a transaction; publish real-time update; run anti-sniping if within threshold.
- Pre-authorization on buyer card via Stripe (configurable min hold or proportional).



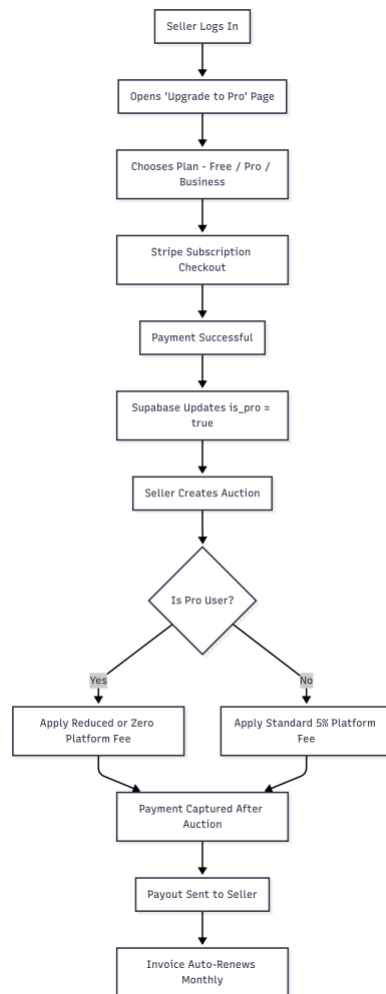
4.1.4 **Auction Close:** CRON or timer finalizes; winner determined; idempotent write to auction outcome.

4.1.5 Payment Capture & Buyer Protection:

- Capture Payment Intent for winner.
- Move funds to Connect balance; hold remainder; deduct 5 platform fee.
- Release payout to seller on buyer confirmation or auto after X days; disputes pause payout.

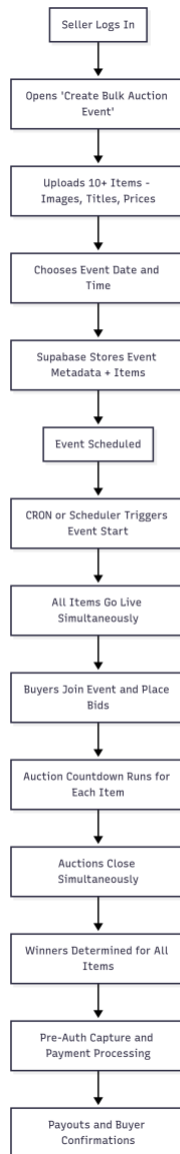
4.1.6 Subscriptions:

- User chooses the option to upgrade to pro or business plan.
- Skips platform fee flow during checkout



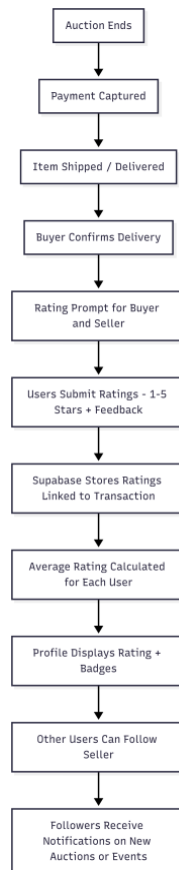
4.1.7 Bulk Auction Event:

- Sellers list multiple items → schedule event → trigger goes live
- Multiple items auctioned in parallel under one event
- Each item follows the normal pre-auth → capture → payout cycle



4.1.8 Rating & Follow System (Post-Transaction Trust):

- Ratings happen after delivery confirmation
- Ratings stored per transaction → averaged per user
- Followers notified of future auctions → building engagement loop



4.1.9 **Disputes:** in-app ticket, evidence upload, support decision tree, partial/full refund handling.

4.1.10 **Messages:** alerts, tracking of sharing of sensitive information shared

The lifecycle of a transaction on SingBid follows this sequence:

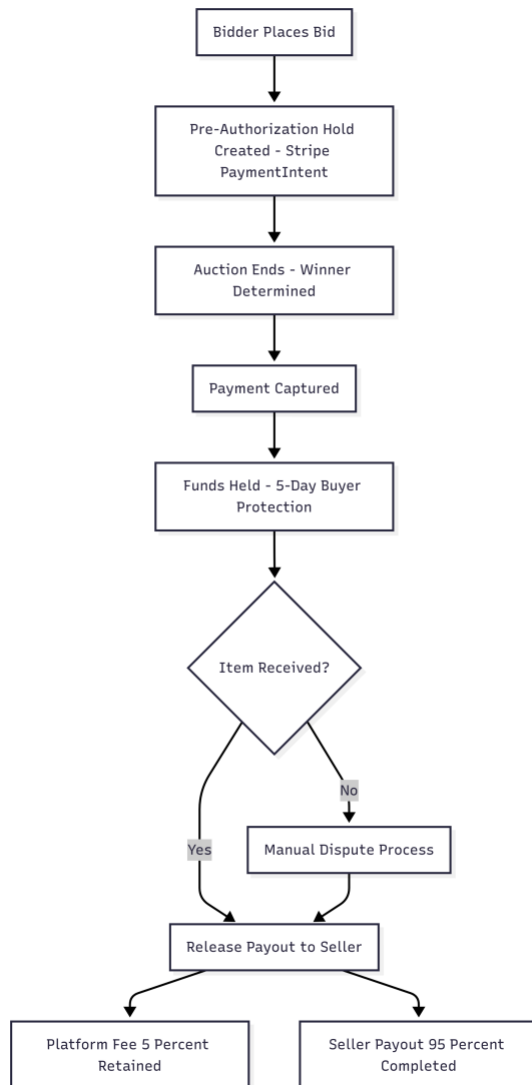
Step 1. Bid Placement: Pre-authorization of funds via PaymentIntent.

Step 2. Auction Finalization: Winner determined; capture occurs.

Step 3. Buyer Protection: Funds held in platform balance (Stripe Account).

Step 4. Payout: Released after 5 days or upon buyer confirmation of delivery.

Step 5. Dispute Handling: Payout frozen until manual resolution.



5. Data Model (Core Tables)

Tables & Database structure in database (Subject to changes & alterations during development, testing & post-production launch, subsequent changes to database will be reflected in this document in subsequent versions.

Table	Key Purpose	Relationship
users	Stores basic account info and role	1 → 1 with user_profiles, 1 → many auctions, subscriptions
auctions	Individual auction listings	many → 1 auction_events, many → 1 users
auction_events	Bulk scheduled auctions	1 → many auctions
bids	Individual bid records	many → 1 auctions
outcomes	Auction results	1 → 1 auctions, 1 → 1 bids
payments	Buyer payment records	many → 1 users
payouts	Seller payout records	many → 1 users
subscriptions	Active recurring plans	many → 1 users
subscription_plans	Metadata for plans	1 → many subscriptions
ratings	Feedback and reputation system	many → 1 users
followers	Social features	many → many (self-reference users)
notifications	Engagement and alerts	many → 1 users

5. Security Controls

- RLS everywhere, least-privilege policies.
- Edge rate limiting + CAPTCHA on auth/bid spikes.
- Stripe signature verification; idempotent webhook keys.
- Image/file validation; signed URLs; optional AV scan.
- PDPA: data minimisation, retention, incident response.
- Admin app isolated with role claims + IP allowlist + WebAuthn.

6. Scaling & Performance

- Horizontal scaling of Edge Functions; DB indexes on (auction_id, created_at, amount).
- Realtime channels per auction; shard popular auctions if needed.
- Caching: static pages on CDN; no-cache on live auction states.
- Load testing with k6; target thousands of concurrent connections per hot auction.

7. Migration from Base44 to Supabase

- Schema mapping; ETL scripts; dual-write (optional) and final cutover.
- Backups and rollback plan; data validation checklist.

8. Monitoring & Ops

- Dashboards: error rate, bid latency, capture success, payout lag.
- Alerts: failed webhooks, abnormal bid spikes, DB locks.
- Runbooks: incident severity matrix; rollback and hotfix playbooks.

9. Compliance Notes

- Keep auction as sale-of-goods (no pay-to-bid). Avoid GCA scope.
- Payments via licensed PSP (Stripe). GST on platform fees when applicable.
- Prohibited-items policy enforced; property/vehicles require guidance.

10. Signatures

Client: _____ Date: 29th September 2025

Vendor:  _____ Date: 29th September 2025